

UNIVERSIDAD EAFIT

Estructuras de Datos y Algoritmos I

Práctica 1 — Estructuras Unidimensionales

Informe técnico

Integrantes

Luisa María Cuervo Prado
Carlos Mario Monsalve Tangarife
Sheryl Astrid Murillo Duque

Repositorio: ADA_P1_Cuervo1_Monsalve2_Murillo3
Lenguaje: C++17

2. Introducción

Este informe documenta la solución desarrollada por el equipo para la Práctica 1 del curso Estructuras de Datos y Algoritmos I, centrada en dos estructuras unidimensionales elementales — la pila (LIFO) y la cola (FIFO) — aplicadas a dos escenarios de la computación real: un sistema de deshacer/rehacer (Undo/Redo) para un editor de código, y el búfer de recepción con limitador de tasa de un firewall. Ambas estructuras se implementaron desde cero en dos representaciones internas (arreglo dinámico y lista enlazada), sin recurrir a los contenedores de la biblioteca estándar de C++ (`std::stack`, `std::queue`, `std::deque`, `std::list`), en cumplimiento estricto de las restricciones del enunciado.

El documento recoge el diseño de las soluciones, su fundamentación teórica, el pseudocódigo de las operaciones fundamentales, el análisis formal de complejidad temporal y espacial (incluyendo el argumento amortizado exigido para el redimensionamiento de la pila y la purga de la cola de marcas de tiempo), los casos de prueba obligatorios, la experimentación con instancias sintéticas de tamaño creciente y la comparación entre la complejidad teórica deducida y el comportamiento medido.

3. Contexto de los dos problemas

Problema 1 — Undo/Redo de un editor de código

Un editor de texto o código debe permitir deshacer (UNDO) la última edición no deshecha y rehacer (REDO) la última edición deshecha, respetando la regla de que toda nueva edición posterior a uno o más UNDO invalida el historial de rehacer. El orden de reversión es estrictamente inverso al de aplicación — el comportamiento LIFO por definición —, por lo que la estructura natural es una pila; en este caso, dos pilas: una de deshacer y una de rehacer.

Problema 2 — Búfer y limitador de tasa de un firewall

Un firewall recibe un flujo de paquetes caracterizados por marca de tiempo y tamaño. Debe almacenarlos temporalmente en un búfer de capacidad fija C , respetando el orden de llegada (FIFO), y debe además aplicar un limitador de tasa por ventana deslizante: un paquete solo se acepta si en los últimos T milisegundos no han llegado ya L o más paquetes. El patrón de insertar por el final y descartar sistemáticamente desde el frente los elementos que salen de la ventana de tiempo es, de nuevo, el comportamiento canónico de una cola.

4. Fundamentación teórica

Pila (LIFO)

Una pila es un TAD con disciplina Last-In-First-Out: el último elemento insertado es el primero en salir. Sus operaciones fundamentales son `push` (insertar en el tope), `pop` (remover y devolver el elemento del tope), `peek/top` (consultar el tope sin removerlo) e `isEmpty`. Sobre arreglo dinámico, el tope corresponde al último índice ocupado; cuando el arreglo se llena, se define una condición de "pila llena" transitoria que dispara un redimensionamiento (en este proyecto, duplicación de la capacidad) en vez de rechazar la operación, ya que el enunciado exige un TAD sin límite fijo de tamaño. Sobre lista enlazada simple, el tope corresponde a la cabeza de la lista; no existe noción de capacidad ni de redimensionamiento, a costa de un puntero adicional por nodo. Más allá del caso de este trabajo, las pilas modelan de forma natural la pila de llamadas de un programa, la evaluación de expresiones y el balanceo de paréntesis, el algoritmo de backtracking, y la navegación "atrás" de un navegador web.

Cola (FIFO)

Una cola es un TAD con disciplina First-In-First-Out. Sus operaciones fundamentales son `enqueue` (insertar al final), `dequeue` (remover y devolver el elemento del frente), `front` (consultar el frente) e `isEmpty`. Sobre arreglo, una implementación ingenua que desplace todos los elementos en cada `dequeue` sería $O(n)$; por eso se usa un arreglo circular con dos índices, `head` y `tail`, que avanzan en aritmética modular, evitando desplazamientos. La condición de "cola llena" y "cola vacía" no puede distinguirse únicamente comparando `head` con `tail`, porque ambos casos pueden coincidir en los mismos índices (un arreglo circular vacío y uno completamente lleno, con capacidad C y `tail` habiendo dado una vuelta completa, comparten `head==tail`); por ello este proyecto mantiene un contador explícito (`count`) que desambigua ambos casos en $O(1)$. Sobre lista enlazada simple, se mantienen punteros a `head` y a `tail`: `enqueue` inserta después de `tail` ($O(1)$) y `dequeue` remueve desde `head` ($O(1)$), sin la ambigüedad de la representación circular ni límite de capacidad. Las colas modelan colas de impresión, colas de tareas (schedulers), búsqueda en amplitud (BFS) y, como en este trabajo, búferes de red.

Listas enlazadas (transversal a ambos problemas)

Una lista enlazada simple se compone de nodos, cada uno con un valor y una referencia (puntero) al siguiente nodo; la estructura se recorre desde una cabeza. A diferencia de un arreglo, no ofrece acceso indexado $O(1)$ — buscar el i -ésimo elemento requiere recorrer i nodos, $O(n)$ —, pero permite inserción y eliminación en $O(1)$ en los extremos donde se mantiene un puntero (cabeza para la pila; cabeza y cola para la cola enlazada), sin necesidad de desplazar elementos ni de redimensionar. Su costo adicional es el overhead de

memoria de un puntero por nodo y la pérdida de localidad espacial: los nodos no son contiguos en memoria, por lo que el recorrido genera más fallos de caché que un arreglo equivalente.

Distinción TAD vs. implementación

Un Tipo Abstracto de Datos define un conjunto de operaciones y su contrato (qué hace cada operación y con qué complejidad esperada), independientemente de cómo se almacenen los datos internamente. Este proyecto expone explícitamente esa distinción: la interfaz de Pila (push/pop/peek/isEmpty/size) se satisface indistintamente con StackArray (arreglo dinámico) y StackList (lista enlazada), y la lógica de negocio de Undo/Redo (GestorUndoRedo<PilaT>) se escribió una sola vez como plantilla de C++, parametrizada por la representación de pila, sin conocer ni depender de si PilaT es un arreglo o una lista. De forma análoga, la interfaz de Cola (enqueue/dequeue/front/isEmpty/size) se satisface con QueueCircular (arreglo circular de capacidad fija) y QueueList (lista enlazada); en el Problema 2, ambas representaciones conviven dentro de la misma solución cumpliendo roles distintos — QueueCircular para el búfer de capacidad fija C, y QueueList para la cola de marcas de tiempo, cuyo tamaño relevante varía con la purga —, lo que constituye evidencia concreta de que ambas satisfacen el mismo contrato desde la perspectiva de quien las usa (RateLimiter), sin exponer su representación interna.

5. Diseño de las soluciones (TAD e implementaciones)

Problema 1

StackArray administra un bloque de memoria crudo (Operacion* mediante new[]/delete[]) con los campos tamaño y capacidad; al llenarse, redimensionar() duplica la capacidad, reserva un nuevo bloque, copia los elementos existentes y libera el bloque anterior. StackList mantiene únicamente un puntero a la cabeza (NodoPila*) y un contador; cada push reserva un nodo y lo enlaza como nueva cabeza. Ambas clases prohíben la copia (constructor y operator= eliminados) porque administran memoria dinámica manualmente y una copia superficial produciría doble liberación.

GestorUndoRedo<PilaT> encapsula dos pilas (pilaUndo, pilaRedo) del mismo tipo PilaT, instanciada explícitamente para StackArray y para StackList (misma lógica, dos representaciones). registrarEdicion aplica la edición sobre el documento (std::string, estructura auxiliar permitida por el enunciado), apila la operación en pilaUndo y vacía por completo pilaRedo. deshacer hace pop de pilaUndo, revierte la operación sobre el documento usando el contenido previo capturado en el momento de la edición, y apila la operación en pilaRedo; rehacer es el proceso inverso.

Problema 2

QueueCircular<T> y QueueList<T> se implementaron como plantillas genéricas de C++ para poder reutilizarse con distintos tipos de dato (size_t para el búfer, uint64_t para las marcas de tiempo) sin duplicar código. QueueCircular usa aritmética modular sobre head y tail y un contador count que desambigua "llena" de "vacía". QueueList mantiene head y tail con inserción en O(1) en ambos extremos relevantes.

RateLimiter combina una QueueCircular<size_t> (buffer_queue, capacidad fija C, el búfer físico de recepción) y una QueueList<uint64_t> (timestamp_queue, la ventana deslizante de marcas de tiempo). Ambas colas crecen siempre en sincronía: cada paquete aceptado se encola en las dos al mismo tiempo, de modo que, cuando purgeExpired() saca de timestamp_queue una marca anterior a t - T, también libera el elemento correspondiente de buffer_queue. Sin esta purga sincronizada, buffer_queue se llenaría una única vez con los primeros C paquetes y rechazaría todo el tráfico restante indefinidamente, lo cual no representa el comportamiento de un firewall real.

Sobre el orden de verificación: processPacket primero purga las marcas expiradas (para que la ventana de tasa nunca quede desactualizada), luego evalúa si hay cupo físico en el búfer (buffer_queue.isFull()) y, solo si lo hay, evalúa la política de tasa (timestamp_queue.size() >= L); el paquete se acepta únicamente si supera ambas validaciones. Verificar primero la restricción física (espacio en el búfer) y después la restricción de política (tasa) refleja el orden natural en que un firewall real descartaría tráfico: no tiene sentido evaluar una política sobre un paquete que de todas formas no podría almacenarse. La restricción C, T y L se recibe como parámetro del constructor de RateLimiter y nunca queda codificada de forma fija, conforme exige el enunciado.

6. Pseudocódigo de las operaciones fundamentales

6.1 Pila sobre arreglo dinámico

```
FUNCION push(P, valor)
  SI tamaño(P) = capacidad(P) ENTONCES
    nuevaCapacidad <- capacidad(P) * 2
    nuevoArreglo <- reservar memoria(nuevaCapacidad)
    COPIAR P.arreglo -> nuevoArreglo (tamaño(P) elementos)
    liberar P.arreglo ; P.arreglo <- nuevoArreglo ; P.capacidad <- nuevaCapacidad
  FIN SI
  P.arreglo[tamaño(P)] <- valor ; tamaño(P) <- tamaño(P) + 1
FIN FUNCION
```

```

FUNCION pop(P)
  SI isEmpty(P) ENTONCES RETORNAR (exito=false)
  tamano(P) <- tamano(P) - 1 ; RETORNAR (exito=true, valor=P.arreglo[tamano(P)])
FIN FUNCION

FUNCION peek(P) // analoga a pop, sin decrementar tamano
FUNCION isEmpty(P) RETORNAR tamano(P) = 0

```

6.2 Pila sobre lista enlazada

```

FUNCION push(P, valor)
  nuevoNodo <- reservar memoria(Nodo) ; nuevoNodo.valor <- valor
  nuevoNodo.siguiente <- P.tope ; P.tope <- nuevoNodo ; P.tamano <- P.tamano + 1
FIN FUNCION

FUNCION pop(P)
  SI isEmpty(P) ENTONCES RETORNAR (exito=false)
  nodoAEliminar <- P.tope ; valor <- nodoAEliminar.valor
  P.tope <- nodoAEliminar.siguiente ; liberar memoria(nodoAEliminar) ; P.tamano <- P.tamano - 1
  RETORNAR (exito=true, valor=valor)
FIN FUNCION

FUNCION isEmpty(P) RETORNAR P.tope = NULO

```

6.3 Lógica de Undo/Redo (independiente de la representación de la pila)

```

FUNCION registrarEdicion(tipo, posicion, contenido)
  contenidoAnterior <- capturar estado actual del documento en 'posicion'
  aplicar la edicion (INSERT/DELETE/REPLACE) sobre 'documento'
  op <- Operacion(tipo, posicion, contenido, contenidoAnterior)
  push(pilaUndo, op) ; vaciarPila(pilaRedo) // toda edicion nueva invalida el redo
FIN FUNCION

FUNCION deshacer()
  (exito, op) <- pop(pilaUndo)
  SI NO exito ENTONCES RETORNAR no-op
  revertir 'op' sobre 'documento' (usar contenidoAnterior) ; push(pilaRedo, op)
FIN FUNCION

FUNCION rehacer() // simetrica a deshacer, usando pilaRedo y contenidoNuevo

```

6.4 Cola circular sobre arreglo (búfer de recepción)

```

FUNCION enqueue(C, valor)
  SI tamano(C) = capacidad(C) ENTONCES RETORNAR ERROR "Cola llena"
  C.datos[tail] <- valor ; tail <- (tail + 1) MOD capacidad ; tamano(C) <- tamano(C) + 1
FIN FUNCION

FUNCION dequeue(C)
  SI isEmpty(C) ENTONCES RETORNAR ERROR "Cola vacia"
  valor <- C.datos[head] ; head <- (head + 1) MOD capacidad ; tamano(C) <- tamano(C) - 1
  RETORNAR valor
FIN FUNCION

FUNCION isFull(C) RETORNAR tamano(C) = capacidad(C)
FUNCION isEmpty(C) RETORNAR tamano(C) = 0

```

6.5 Purga de marcas expiradas y control del firewall

```

FUNCION purgarExpirados(colaMarcas, colaBuffer, tActual, T)
  MIENTRAS NO isEmpty(colaMarcas) Y front(colaMarcas) < (tActual - T) HACER
    dequeue(colaMarcas) ; dequeue(colaBuffer) // ambas colas se purgan en sincronia
  FIN MIENTRAS
  // borde estricto: un timestamp exactamente en t0 + T NO se purga
FIN FUNCION

FUNCION procesarPaquete(timestamp, tamanoBytes)
  purgarExpirados(marcasTiempo, buffer, timestamp, T) // PASO 1
  SI isFull(buffer) ENTONCES RETORNAR "RECHAZADO: buffer lleno" // PASO 2
  SI tamano(marcasTiempo) >= L ENTONCES RETORNAR "RECHAZADO: limite de tasa" // PASO 3
  enqueue(buffer, paquete) ; enqueue(marcasTiempo, timestamp) // PASO 4
  actualizar estadísticas (aceptados, ocupacion maxima)
  RETORNAR "ACEPTADO"
FIN FUNCION

```

7. Análisis de complejidad

7.1 Tabla de complejidad — Problema 1 (Pila)

Operación	Mejor caso	Caso prom.	Peor caso	Complej. espacial	Justificación
push (arreglo, sin redimensionar)	$\Omega(1)$	$\Theta(1)$	$O(1)$	$O(1)$	Escritura en datos[tamano] e incremento de índice; sin bucles ni copias.

Operación	Mejor caso	Caso prom.	Peor caso	Complej. espacial	Justificación
push (arreglo, con redimensionamiento)	$\Omega(1)$	$\Theta(1)$ amortizado	$O(n)$ puntual	$O(n)$ transitorio	El evento de copia cuesta $O(n)$ puntual; por análisis agregado, n pushes con duplicación cuestan en total $O(n)$ (serie $1+2+4+\dots+n < 2n$), por lo que el costo amortizado por push es $\Theta(1)$.
push (lista enlazada)	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$O(1)/\text{nodo}$, $O(n)$ total	Reservar un nodo y enlazarlo como nueva cabeza; no depende del tamaño actual.
pop / peek (ambas repres.)	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$O(1)$	Acceso/eliminación directa sobre el extremo activo, sin recorrido.
isEmpty	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$O(1)$	Comparación directa (tamaño = 0 ó tope = NULO).

7.2 Tabla de complejidad — Problema 2 (Cola)

Operación	Mejor caso	Caso prom.	Peor caso	Complej. espacial	Justificación
enqueue (cola circular)	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$O(1)/\text{op}$, $O(C)$ total	Escritura en datos[tail] y actualización modular de tail; costo fijo, independiente de C .
dequeue (cola circular)	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$O(1)$	Avance modular de head y decremento de count; no hay corrimiento de elementos.
Purga de marcas expiradas (por paquete)	$\Omega(1)$	$\Theta(1)$ amortizado	$O(k)$ puntual ($k = \text{expiradas}$)	$O(1)$ auxiliar	Cada marca se encola una vez y se purga a lo sumo una vez en todo el procesamiento; por análisis agregado, el costo total de purgas en n paquetes es $O(n) \Rightarrow \Theta(1)$ amortizado por paquete.
isFull / isEmpty	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$O(1)$	Contador explícito (count) distingue "llena" de "vacía" sin ambigüedad de head = tail.

7.3 Análisis de complejidad global del problema completo

Problema 1 (n eventos). Cada operación de pila (push/pop) es $O(1)$ amortizado, sin importar la representación. Sin embargo, cada EDIT también se aplica sobre el documento simulado mediante `std::string::insert/erase`, cuyo costo es $O(m)$, con m la longitud actual del documento, porque debe desplazar los caracteres restantes. En el peor caso, con m creciendo proporcionalmente a n , el costo total de procesar el archivo completo es $O(n \cdot m) = O(n^2)$. La complejidad espacial total es $O(n)$ para las dos pilas (cada Operación guarda hasta dos cadenas) más $O(m)$ para el documento, ambas acotadas por $O(n)$.

Problema 2 (n paquetes). Cada paquete dispara una purga $O(1)$ amortizado, dos comparaciones $O(1)$ y, si se acepta, un enqueue $O(1)$ en cada cola. El costo total de procesar n paquetes es $\Theta(n)$ en el peor caso amortizado — lineal. En cuanto al espacio: como `buffer_queue` tiene capacidad fija C y `timestamp_queue` se purga exactamente en sincronía con `buffer_queue`, el tamaño de ambas colas está acotado por C en todo momento, sin importar cuán grande sea n . El espacio total usado es entonces $O(C)$: constante respecto al número de paquetes procesados.

Argumento amortizado. Es obligatorio en el Problema 2 (purga por ventana deslizante) y se recomienda en el Problema 1 cuando se usa duplicación de capacidad; en ambos casos el argumento es de análisis agregado: aunque una operación puntual pueda costar $O(n)$ (una copia grande o una purga de muchos elementos de una vez), cada elemento individual solo puede provocar ese costo un número acotado de veces a lo largo de toda la secuencia de operaciones, de modo que el costo total repartido entre las n operaciones es $O(n)$, es decir $\Theta(1)$ por operación en promedio.

8. Casos de prueba

Se implementaron los 7 casos obligatorios de la Sección 11 del enunciado para cada problema, disponibles en `tests/` con su salida verificada en `results/`. La Tabla 8.1 resume el Problema 1 y la Tabla 8.2 el Problema 2.

8.1 Problema 1 (Undo/Redo)

#	Caso (Problema 1)	Archivo	Resultado observado
1	Secuencia normal mixta (INSERT, UNDO, REDO, INSERT)	caso1_secuencia_normal.txt	Documento final "FINpeoplehola"; Undo=3, Redo=0

#	Caso (Problema 1)	Archivo	Resultado observado
2	UNDO sin ediciones previas	caso2_undo_vacio.txt	"UNDO no-op: pila Undo vacia"; no aborta
3	Una edición, UNDO, REDO	caso3_edicion_undo_redo.txt	Edición revertida y reaplicada correctamente
4	Edición inmediatamente después de un UNDO	caso4_edicion_tras_undo.txt	Redo se vacía tras la nueva edición; REDO posterior es no-op
5	N ediciones seguidas de N deshacer	caso5_n_ediciones_n_deshacer.txt	Las 3 ediciones se deshacen en orden LIFO inverso
6	Más REDO que elementos disponibles	caso6_redo_exceso.txt	Segundo REDO reporta no-op sin fallar
7	Crecimiento de capacidad del arreglo	caso7_crecimiento_capacidad.txt	6 pushes con capacidad inicial 2 fuerzan 2 redimensionamientos; resultado idéntico en ambas representaciones

8.2 Problema 2 (Firewall)

#	Caso (Problema 2)	Archivo	Resultado observado
1	Flujo normal por debajo de C y L	p2_caso1_flujo_normal.txt	3/3 paquetes ACEPTADOS; ocupación máxima 3/5
2	Consulta sobre búfer vacío	p2_caso2_buffer_vacio.txt	Sin paquetes; estado final VACIO en ambas colas
3	Un único paquete	p2_caso3_unico_paquete.txt	1 paquete ACEPTADO; búfer con 1 elemento
4	Búfer exactamente lleno + paquete adicional (C=2)	p2_caso4_buffer_lleno.txt	2 ACEPTADOS, 3.º RECHAZADO BUFFER FULL; búfer LLENO 2/2
5	Ráfaga que excede L dentro de T (L=2)	p2_caso5_rafaga_excede_l.txt	2 ACEPTADOS, 3.º RECHAZADO RATE LIMIT
6	Dequeue/front sobre cola vacía	p2_caso6_dequeue_vacio.txt	QueueCircular y QueueList lanzan std::underflow error, capturada sin caída del programa
7	Paquete exactamente en el borde $t = t_0 + T$ (T=1000)	p2_caso7_borde_tiempo.txt	$t=1100$ con $t_0=100$ y $T=1000$: la marca en t_0 se purga (borde excluido); 2/2 ACEPTADOS

Adicionalmente, los 7 casos del Problema 1 se ejecutaron sobre las dos representaciones de la Pila (StackArray y StackList) usando la misma lógica de GestorUndoRedo<PilaT>: los archivos de salida resultantes son idénticos byte a byte en los 7 casos (results/comparacion_pilas.txt), confirmando que ambas representaciones satisfacen el mismo contrato de interfaz. Para la Cola, la equivalencia de contrato entre QueueCircular y QueueList se evidencia de forma estructural: ambas se usan simultáneamente dentro de RateLimiter (Sección 5) exponiendo la misma interfaz (enqueue/dequeue/front/isEmpty/size) sobre representaciones internas distintas.

9. Experimentación

Los datos sintéticos se generan de forma determinista con semilla fija (semilla = 42, data/generador.cpp), para tamaños $n \in \{1\,000; 10\,000; 100\,000; 1\,000\,000\}$. Para el Problema 2 se usaron los parámetros $C=10$, $T=1000$ ms, $L=5$. Cada tamaño se ejecutó 5 veces con std::chrono::high_resolution_clock, reportando media y desviación estándar del tiempo total, medidos en el equipo del integrante a cargo de las pruebas (Ryzen 5 3600, 32 GB RAM 3200 MHz, almacenamiento M.2 NVMe), compilando con g++ -std=c++17 -O2.

9.1 Problema 1 — tiempo total de procesamiento

n (eventos)	Tiempo promedio (ms)	Desviación estándar (ms)
1 000	2.00	0.44
10 000	15.11	0.85
100 000	350.47	12.44
1 000 000	47 259.30	302.89

Métrica de dominio: en el archivo de $n=1\,000\,000$ eventos, la proporción aproximada generada por el generador sintético es 70 % EDIT, 15 % UNDO y 15 % REDO (distribución fija en data/generador.cpp), lo que produce una mezcla representativa de ediciones efectivas y no-ops.

9.2 Problema 2 — tiempo total de procesamiento

n (paquetes)	Tiempo promedio (ms)	Desviación estándar (ms)
1 000	0.0046	0.0004
10 000	0.0488	0.0021
100 000	0.5254	0.0379
1 000 000	5.3153	0.3520

Métrica de dominio: para cada tamaño se registran los contadores de total_accepted, total_rejected_buffer y total_rejected_rate expuestos por RateLimiter, además de la ocupación máxima del búfer, disponibles en las salidas de results/p2_caso*_salida.txt para los casos de prueba dedicados.

10. Comparación teórico-experimental

Problema 2 — consistente con $O(1)$ amortizado. El tiempo total crece de forma prácticamente lineal con n : al multiplicar n por 10 (de 100 000 a 1 000 000), el tiempo se multiplica por $\approx 10.1\times$, tal como predice el análisis de la Sección 7.3 ($\Theta(n)$ total, $O(1)$ amortizado por paquete). Esto confirma que la purga sincronizada de `buffer_queue` y `timestamp_queue` no introduce un costo oculto superlineal.

Problema 1 — crecimiento superlineal explicado por el documento, no por la pila. El tiempo NO crece linealmente: de 100 000 a 1 000 000 eventos ($10\times$), el tiempo se multiplica por $\approx 135\times$. Esto es consistente con la Sección 7.3: la pila en sí sigue siendo $O(1)$ amortizado, pero cada EDIT se aplica sobre `std::string` mediante `insert/erase`, cuyo costo depende de la longitud m del documento en ese momento (debe desplazar los caracteres posteriores a la posición editada). Como el documento simulado crece con el número de inserciones netas, el costo por edición aumenta con n , y el costo total se aproxima a $O(n\cdot m)$; el crecimiento superlineal observado es evidencia experimental directa de ese término, no un defecto de la implementación de la pila.

Efecto de constantes ocultas y de la jerarquía de memoria. Para n pequeño (1 000), la desviación estándar relativa es más alta ($\approx 22\%$ en el Problema 1) porque el tiempo total es comparable al ruido del sistema operativo y del reloj; esto justifica la recomendación del enunciado de usar n en el orden de 10^5 – 10^6 . En términos generales, un arreglo contiguo (`StackArray`, `QueueCircular`) favorece la localidad espacial de acceso frente a una lista enlazada, cuyos nodos están dispersos en el heap y generan más fallos de caché por operación, aunque ambas representaciones sean $O(1)$ por operación en notación asintótica; en este proyecto la medición de tiempos se realizó sobre las representaciones de arreglo (`StackArray`, `QueueCircular+QueueList`), mientras que la comparación entre representaciones (Sección 8.2) se limitó a verificar equivalencia de resultados, no de tiempos.

Condiciones de ejecución y evidencia de comportamiento amortizado. Todas las mediciones se realizaron con optimización -O2 en el mismo equipo, con 5 repeticiones por tamaño. El caso de prueba dedicado al crecimiento de capacidad (caso 7, capacidad inicial 2 sobre 6 pushes) provoca dos eventos de redimensionamiento puntuales; su costo queda absorbido dentro del promedio de la Sección 9.1 sin generar una desviación estándar anómala, consistente con el argumento amortizado de la Sección 7.3.

11. Conclusiones

- La pila y la cola, implementadas desde cero en dos representaciones cada una, resolvieron ambos problemas cumpliendo el mismo contrato de TAD sin exponer su representación interna a la lógica de negocio (`GestorUndoRedo<PilaT>` y `RateLimiter`).
- El comportamiento $O(1)$ amortizado de push (con duplicación de capacidad) y de la purga de marcas expiradas se confirma tanto por el análisis teórico agregado como por la ausencia de crecimiento superlineal en el Problema 2.
- El crecimiento superlineal medido en el Problema 1 no proviene de la pila, sino del costo $O(m)$ de aplicar cada edición sobre el documento simulado (`std::string::insert/erase`); esta distinción solo es visible al separar la complejidad de la estructura de datos evaluada de la de las estructuras auxiliares permitidas por el enunciado.
- Sincronizar la purga del búfer con la purga de la cola de marcas de tiempo fue la decisión de diseño central del Problema 2: sin ella, el búfer se saturaría de forma permanente tras los primeros C paquetes, sin reflejar el comportamiento de un firewall real, y además mantiene el espacio usado acotado en $O(C)$ sin importar cuán grande sea el flujo de tráfico procesado.

12. Uso de herramientas de Inteligencia Artificial

El registro detallado de las interacciones con herramientas de IA utilizadas durante el desarrollo de esta práctica (consultas conceptuales, aclaración de errores de compilación, retroalimentación sobre código ya escrito por el equipo y apoyo en la redacción del informe) se presenta al final de este documento en las imágenes con los prompts usados por cada uno de los estudiantes.

13. Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4.^a ed.). MIT Press.
- `cppreference.com`. (s. f.). C++ Standard Library reference — `std::string`, `std::chrono`. Recuperado de <https://en.cppreference.com>
- Universidad EAFIT. (2026). Enunciado: Práctica 1 — Estructuras Unidimensionales. Estructuras de Datos y Algoritmos I.
- Weiss, M. A. (2013). *Data Structures and Algorithm Analysis in C++* (4.^a ed.). Pearson.
- WWW.youtube.com/programacionats.

14. Contribución individual de los integrantes

Integrante	Contribución
Luisa María Cuervo Prado — Problema 1	Diseño e implementación de la Pila sobre arreglo dinámico (StackArray) y sobre lista enlazada (StackList), y de la lógica de Undo/Redo (GestorUndoRedo<PilaT>), incluyendo la política de crecimiento por duplicación y el manejo de casos límite del Problema 1 (pila Undo/Redo vacía, invalidación del Redo tras nueva edición).
Sheryl Astrid Murillo Duque — Problema 2	Diseño e implementación de la Cola circular sobre arreglo (QueueCircular) y de la Cola sobre lista enlazada (QueueList), y de la lógica del firewall (RateLimiter): sincronización de la purga entre el búfer y la cola de marcas de tiempo, y manejo de los casos límite del Problema 2 (búfer lleno, ráfaga que excede la tasa, borde $t = t_0 + T$).
Carlos Mario Monsalve Tangarife — Tester	Diseño y ejecución de los 14 casos de prueba obligatorios (Sección 11) y del generador de datos sintéticos con semilla fija (data/generador.cpp); ejecución de la experimentación de tiempos de ambos problemas (5 repeticiones por tamaño, cálculo de media y desviación estándar) en su equipo personal (specs en el README), y verificación de la equivalencia de resultados entre representaciones.

12. Uso de herramientas de Inteligencia Artificial (Imágenes)

Prompt usados en la IA por Carlos Monsalve / QA

Hola, actúa como un profesor de Estructura de Datos y Algoritmos con alto conocimiento en las EDA pilas y colas, requiero que me definas de manera simple como debería ser la implementación de estas estructuras en C++ sin una biblioteca, ya que requiero revisar que estén bien implementadas en mi práctica de la materia.

como genero archivos de prueba con una semilla fija en c++ para que los experimentos me den siempre igual

como hago un caso de prueba en c++ para verificar que el buffer de la cola se comporte bien cuando se llena del todo

me puedes ayudar a construirlo poco a poco haciendome preguntas para crearlo

dame un comando de g++ en terminal para compilar todos los archivos cpp de la carpeta src sin que me de error de main duplicado

El error de `main duplicado` ocurre porque tienes múltiples archivos `.cpp` en la carpeta `src/` que contienen su propia función `main()` (muy común cuando guardas pruebas o algoritmos diferentes en el mismo proyecto).

Si le pides a `g++` que compile `src/*.cpp` todo junto, intentará empaquetarlos en un solo ejecutable y la etapa de enlazado (*linker*) fallará.

por que el tiempo del programa de texto se dispara tanto en un millón de datos si las operaciones de la pila son $O(1)$

El disparo en el tiempo no se debe a la complejidad algorítmica ($O(1)$) de las operaciones de la pila, sino al I/O (Entrada/Salida) y al comportamiento de la memoria física.

Aunque `push` y `pop` tomen tiempo constante $O(1)$, cuando ejecutas un millón de operaciones procesando texto, los cuellos de botella se mueven a nivel de hardware y del sistema operativo.

Prompt usados en la IA por Sheryl Murillo / P2

Yo usaría la cola circular porque como ya sabemos cuál es la capacidad máxima C , dejamos el espacio fijo de entrada y nos ahorramos el lío de estar pidiendo y liberando memoria a cada rato. Además, para un rate limiter es super eficiente.

en la cola circular guardo el struct con timestamp y bytes, por q cuando me toca sacar uno viejo con el `pop()` necesito saber exactamente cuantos bytes restarle a la suma pa no embolatarme

Que crees que le hace falta a este pseudocódigo del main)
Procedimiento ejecutarPruebaP2(archivoEntrada, C, T, L):
Abrir archivoEntrada para Lectura
Si El archivo NO existe Entonces:

necesito que me ayudes a entender la logica de esta practica para poder construir mi código

Parce yo haria las validaciones en este orden:

Primero purgo los viejos que ya salieron de la ventana T para liberar espacio.

me sale este error, ayudame a entenderlo `g++ -std=c++17 -O2 src/main.cpp src/undoredo.cpp src/stack_array.cpp src/ratelimiter.cpp src/queue_circular.cpp src/queue_list.cpp -o ada_p1`

me puedes recomendar un video en youtube para entender como puedo hacer el código con colas, etc

ayudame a solucionar, esto me sale en la terminal `g++ -std=c++17 -O2 src/main.cpp src/undoredo.cpp src/stack_array.cpp src/ratelimiter.cpp src/queue_circular.cpp src/queue_list.cpp -o ada_p1`

Prompt usados en la IA por Luisa Cuervo / P1

Para mí tiene sentido, yo creo así se sabe donde estaba (posicion9 para borrarlos y : se desea realizar un Redo después. Una pregunta ese código haría parte del main.cpp, no?

Me sale este problema con size declaration is incompatible with "int StackList::size()" (declared at line 22)

Voy a plantear la idea y me dices si está bien el constructor se pondría la inicialización del arreglo, podríamos usar un for que lo recorra y en el destructor la parte de delete que libera la pila

En el constructor sería guardar la capacidad, poner el tamaño en 0 y revisar el arreglo con new[] usando esa capacidad y el destructor con delete[]

Pop entonces sería que verificamos que el arreglo sea mayor que 0, se decrementa el tamaño del mismo no se usa delete para este caso, básicamente se dejaría así

idea para manejar las operaciones de la pila cuando está vacía que creo que sería bueno usar boolean, para todos como en pop tipo: pila = [10, 20, 30] pop(elemento) elemento = 30 resultado = true, mi idea es que si la pila no está vacía y se puede realizar la operación se usa boolean para marcar el programa